Tsampi BFT: leaderless voting with quorum-ancestry finality

Published August 17, 2026
Updated November 1, 2026

Tim Watts
tim@readevalprint.com

Abstract

Tsampi BFT is a leaderless, asynchronous Byzantine fault tolerant state replication protocol. A Vote endorses one Block and uses PrevVote and PrevSelfVote to record ancestry and signed order. Blocks contain Votes endorsing their parent Block, and different children can contain the same Votes. Ancestry, scores and historical quorum queries determine permitted Votes and finalization. Under fixed authority and a quorum for each parent, three later Blocks finalize a selected Block; a fourth finalizes its child. The paper states the assumptions for Safety and conditional progress and compares related research.

1 Introduction

Tsampi BFT is a leaderless, asynchronous Byzantine fault tolerant state replication protocol. Safety requires compatible finalized lineages across honest nodes and time. Liveness requires every honest node to eventually finalize a Block above any specified common reference height. Validators sign Votes, and proposed Blocks contain Votes endorsing their parent Block. PrevVote follows Block ancestry, while PrevSelfVote records one Validator's signed order within active linked history. Nodes use ancestry, scores and quorum queries to select Blocks and advance their finalized Blocks.

2 Votes, Blocks and Validators

2.1 Validators and stake

An authority specifies the Validators, their stake, the chain identity and the endorsement threshold for admitting a child Block. A Validator has positive stake under that authority. A node retains and checks Blocks and Votes; it may sign for a Validator or observe without signing. For authority s with Validator set V_s, let $w_s(i)$ denote Validator i's stake and W_s its total stake.

$$W_s = \sum_{i \in V_s} w_s(i).$$

Each check uses the authority recorded by the relevant Block, preserving historical weights after later exclusions.

2.2 Blocks and parent Votes

A non-genesis Block identifies one exact parent Block and contains Votes endorsing that parent under its historical authority. Each signing Validator occurs once, and the Block references the proposing Validator's parent Vote as its proposer Vote. The proposing Validator signs a separate detached Vote endorsing the completed child Block. Requiring this Vote first makes each proposer begin with its own Block and restricts later endorsements to permitted improvements. Different children can contain the same parent Votes, including children proposed by one Validator. The genesis Block has height zero, empty parent, proposer Vote and inherited Q3 references, and empty Vote and evidence collections.

For N Validators following the improvement rules, consider one child Block each, with a common parent and equal Q1 and Q2. If initial endorsements were unrestricted, each Validator could endorse all N children from least to most preferred, giving N^2 Votes. Requiring the detached Vote first makes each proposer start with its own child and endorse only better children afterward. From most to least preferred child, proposers can sign at most $1, 2, \ldots, N$ Votes respectively, bounding the total V, including detached Votes.

$$V \le \sum_{r=1}^{N} r = \frac{N(N+1)}{2}.$$

Additional Votes after the detached Votes total at most $N(N-1)/2$. The leading term falls from N^2 to $N^2/2$, reducing signing and verification work for these competing children.

A child Block uses its parent's endorsement threshold n/d, with $0 < n \le d$, to determine the required stake. The default $n/d = 2/3$ requires strictly more than two thirds of the parent's stake; $n = d$ requires all stake. Let U_B be the distinct signing Validators of child B's parent Votes and A_B their total stake under parent P's authority s_P.

$$A_B = \sum_{i \in U_B} w_{s_P}(i),$$

$$\begin{cases} A_B d > W_{s_P} n, & n < d, \\ A_B = W_{s_P}, & n = d. \end{cases}$$

The proposing Validator must have positive stake under that parent authority. Verified evidence determines the child's reduced authority as described in Section 4.1. Each child also advances its parent's raw Q1 result or remaining quorum stake under Section 3.2.

2.3 Votes and references

A Vote identifies its signing Validator and one exact endorsed Block, written $T(v)$ for Vote v. Authentication binds the Vote's contents to that Validator and checks its historical stake. Its chain, height and proposing Validator must match the endorsed Block, with Vote height $\text{height}(T(v)) + 1$. The Vote also carries PrevVote, PrevSelfVote, score context and evidence required by its signed references. A node admits at most one distinct Vote per Validator and exact endorsed Block on a chain.

2.3.1 PrevVote

Tsampi's core logic model is a VoteChain: a chain of Votes linked by PrevVote. Each Block references its proposer Vote, which endorses the parent Block. PrevVote names the inner identity of that proposer Vote, resolved through the exact endorsed Block. The inner identity fixes the signing Validator, chain, height, proposing Validator and PrevVote; the complete Vote additionally identifies its endorsed Block. Following PrevVote therefore follows the signed Votes underlying Block ancestry. A Vote endorsing genesis has an empty PrevVote reference.

2.3.2 PrevSelfVote

During live admission, PrevSelfVote names the preceding signed Vote by the same Validator on the same chain. These references give the Validator's active linked history a total causal order across endorsed Blocks. Each successor endorses a Block that improves on its predecessor's endorsed Block under Section 3.4. The node also checks improvement against the Blocks established by that Vote's referenced history. The linked history remains active while any endorsed Block equals or descends from the new endorsed Block's Q3. Retirement permits an empty PrevSelfVote after a complete check, while recorded history retains every signed Vote.

Catch-up establishes Q3 support from authenticated Votes collected across referenced descendant Blocks, counting distinct Validators under an established historical authority. With strictly more than two thirds honest stake, each quorum includes honest Validators who already validated those descendants and their active histories. Catch-up relies on that prior validation for finalized history under the assumptions in Section 4.2. Fresh Votes after catch-up satisfy the active PrevSelfVote and improvement checks.

3 Block selection and finalization

3.1 Ancestry

A lineage is the exact sequence of parent-linked Blocks from genesis through a specified Block. This sequence determines whether finalized lineages are compatible. Write $A \preceq B$ when A is an ancestor of or equal to B, and $A \prec B$ for strict ancestry. Two lineages are compatible when one is a prefix of the other; otherwise they are incomparable.

3.2 Q1, Q2 and Q3

Q1 scans from an admitted Block's parent toward genesis and accumulates distinct signing Validators from carried parent Votes. A Vote supports its endorsed Block and that Block's ancestors. At each Block T, Q1 counts each accumulated Validator once under T's historical authority. For authority s_T and counted Validator set U, a quorum requires the fixed strict two-thirds inequality below.

$$3 \sum_{i \in U} w_{s_T}(i) > 2W_{s_T}.$$

Q1 returns the Block with the first completed quorum, unless an inherited limiting Block stops the query earlier. An inherited limit or genesis fallback identifies the returned Block without establishing a completed quorum.

Each child must advance its parent's raw Q1 result or increase the remaining distinct stake. This ties Block growth to the quorum queries used for finalization. The parent Block's Q3 height limits the child's Q1 scan; an empty inherited reference permits scanning to genesis. The full query replaces an absent raw result with genesis. Equal raw results require increased remaining distinct stake; different nonempty results require the child's result to strictly descend from the parent's. A nonempty child result also permits progress when the parent's raw result is absent.

Q2 and Q3 repeat Q1 from the Block returned by the preceding query. Each repetition uses that Block's own lineage, historical authority and inherited limit. Each repetition starts with zero counted Validators, as expressed by the following definitions.

$$\mathrm{Q}_2(B) = \mathrm{Q}_1(\mathrm{Q}_1(B)), \qquad \mathrm{Q}_3(B) = \mathrm{Q}_1(\mathrm{Q}_2(B)).$$

The specified Block's ancestry and historical authority make each query reproducible as the node's state advances.

3.3 Finalized Blocks

A node's finalized Block records the ancestry that later finalization must preserve. Compatible Q3 results from admitted Blocks let the node extend that ancestry. The node retains equal or deeper compatible Q3 results from admitted lineages extending its existing finalized Block. Its Tip is an admitted Block selected for possible finalization and must equal or descend from the finalized Block. As finality advances, a compatible Tip may remain; otherwise the node selects the newly finalized Block as Tip. The finalized Block can therefore advance beyond the Tip's own historical Q3.

Consider a fixed authority and lineage G, A, B, C, D, E, with a quorum of parent Votes carried by every child. At D, the queries return the following Blocks.

$$\mathrm{Q}_1(D) = C, \quad \mathrm{Q}_2(D) = B, \quad \mathrm{Q}_3(D) = A.$$

Thus a node admitting D can finalize A after three completed Q1 advances. A fourth advance gives $\mathrm{Q}_3(E) = B$ and lets that node finalize the first new child of A. These advances are asynchronous completion events and can take arbitrary time. The example demonstrates progress beyond A under the delivery and fresh generation assumptions of Section 4.3.

3.4 Block comparison

Tip selection and signing use the same improvement rule for a change of endorsed Block. Normal selection chooses an admitted eligible Block equal to or descending from the node's finalized Block. Signing also preserves the obligations in the Validator's recorded Votes and the order declared through PrevSelfVote. Verified evidence determines the child's authority, and the improvement rule determines permitted Tip changes.

3.4.1 Scores

The proposer score is a fixed input attached to the proposer Vote and reused by every Block referencing that Vote. On one lineage,

prefer the strict descendant. On incomparable lineages, compare proposer Votes at the first different child Blocks and prefer the higher score. For equal scores, prefer the smaller Block identity; later descendants preserve this result. Write $R(A, B)$ when the comparison prefers Block A to Block B.

3.4.2 Improvement

Improvement determines whether new Block N may replace old Block O as a Tip or endorsed Block. On one lineage, improvement requires $O \prec N$. For incomparable Blocks sharing genesis, the permitted alternatives are given below.

$$\begin{aligned}\text{Improves}(N, O) \iff\ & [\mathrm{Q}_2(O) \prec \mathrm{Q}_2(N)] \\ & \vee [R(N, O) \\ & \wedge (\mathrm{Q}_1(O) \preceq \mathrm{Q}_1(N) \\ & \quad \vee \mathrm{Q}_2(O) \preceq \mathrm{Q}_2(N))] \\ & \vee [\mathrm{Q}_2(O) \prec \mathrm{Q}_1(N) \\ & \quad \wedge \neg(\mathrm{Q}_2(N) \prec \mathrm{Q}_1(O))] \\ & \vee [\mathrm{Q}_3(O) \prec \mathrm{Q}_3(N) \\ & \quad \wedge \text{height}(\mathrm{Q}_2(O)) \leq \text{height}(\mathrm{Q}_3(N))].\end{aligned}$$

The first alternative permits strict Q2 advance without a better score. The score alternative requires Q1 or Q2 ancestry in addition to $R(N, O)$. The negated condition retains equality, reverse ancestry and incomparable lineages as the exact complement of strict ancestry.

The final alternative permits an advancing Q3 that reaches the height of the old Q2 Block. Its completed quorum support must satisfy the conditions in Section 4.2.3. Ancestry and score alternatives preserve Q2; this Q3 alternative can preserve finality while Q2 moves to an incomparable lineage. The new Vote retains PrevSelfVote while its earlier linked history remains active.

Suppose children B and X share parent A, proposer Vote p and a quorum of parent Votes. Their scores are equal, and $\mathrm{Q}_1(B) = \mathrm{Q}_1(X) = A$. If B has the smaller identity, the score and Q1 conditions permit improvement from X to B. Votes on distinct Blocks sharing PrevVote follow the PrevSelfVote and active linked history checks for permitted signed order.

4 Safety and Liveness

4.1 Evidence and authority

Evidence identifies violations by a signing Validator through authenticated Votes and their references. Direct evidence includes distinct Votes sharing PrevVote and the same exact endorsed Block, or distinct successors sharing a nonempty PrevSelfVote. Other pairs require mutually active endorsed Blocks and neither Vote in the other's valid bounded linked history. Derived evidence identifies a Vote violating required evidence or direction rules, using its signed references and verified supporting evidence. A fresh Vote includes its required evidence and the available verified evidence. Each Validator's stake is excluded once, even when several verified pieces of evidence identify that Validator.

A child Block records verified evidence and excludes the identified Validators from its authority. For parent authority s_P and identified Validator set E_B, the child's authority is $s_B = s_P \setminus E_B$. Admission and parent Vote counting continue to use s_P; later children use s_B. Historical authority remains unchanged, and s_B must contain at least one Validator.

4.2 Assumptions and Safety

4.2.1 Finalized history

Safety requires compatible finalized histories across honest nodes, while preservation requires each node's later history to extend its earlier history. These conditions make finalized Blocks stable references for later finalization. Let $O(s, i)$ be honest node i's finalized lineage at state s of execution H. Write $L \sqsubseteq R$ when L is a prefix of R. For all honest nodes i, j and states s, t in H, require the following relations.

$$\begin{aligned}\text{Safety}: &\quad O(s, i) \sqsubseteq O(t, j) \ \vee\ O(t, j) \sqsubseteq O(s, i), \\ \text{Preservation}: &\quad s \leq_H t \implies O(s, i) \sqsubseteq O(t, i).\end{aligned}$$

The theorem establishes compatible finalized Blocks within each reachable state under the following assumptions.

4.2.2 Quorum overlap

The Safety argument uses the honest Validator shared by any two quorums. The improvement rule in Section 3.4 preserves Q2 ancestry or requires established Q3 support for catch-up. Assume authenticated Votes, a common genesis and an initial authority with distinct Validators holding positive stake. Honest Validators hold strictly more than two thirds of the initial stake and check improvement against every earlier Vote they recorded. Verified exclusions preserve honest Validators, so quorums under independently reduced historical authorities derived from that initial authority share an honest Validator. The theorem requires a quorum for child Block admission; Section 2.2 defines the protocol's configurable endorsement threshold.

4.2.3 Q3 support for catch-up

The Q3 catch-up rule in Section 3.4 uses history that honest Validators have already validated. For finalized Block F, let C be the Block at which its quorum support is established under the applicable historical authority. Each contributing recorded Vote v must retain F in its Q2 ancestry and endorse a Block below C.

$$F \preceq \mathrm{Q}_2(T(v)), \qquad \text{height}(T(v)) < \text{height}(C).$$

For non-genesis Q3 F, the node must already have admitted recorded Votes from a quorum, each satisfying both inequalities above. Block C must have height at most that of the new endorsed Block. The height bounds tie the new Vote to earlier validated Votes, preserving finalized ancestry during catch-up.

4.3 Conditional progress

Conditional progress assumes that every honest node eventually receives the same finite ordered collection of proposed Block lineages. Its first lineage is valid and precedes every other listed lineage under the Block comparison. Required Blocks and Votes, including references retained by any honest node, must eventually reach every honest node. Delivery permits arbitrary

finite delays, duplicates and reordered messages. This differs from partial synchrony, which adds timing bounds under the conditions studied by Dwork, Lynch and Stockmeyer [1].

Honest Validators must hold strictly more than two thirds of the stake in every applicable parent authority. Each honest Validator supplies a valid Vote in each sequence used to finalize the selected Block and a descendant. These Votes have increasing recorded order, distinct exact endorsed Blocks per Validator, and permitted changes relative to earlier recorded Votes. Each non-genesis Q3 has already admitted support with the height bound in Section 4.2.3. Each contributing Vote satisfies $F \preceq \mathrm{Q}_2(T(v))$ and $\mathrm{height}(T(v)) < \mathrm{height}(C)$ for the Block F being finalized. Fresh generation supplies a valid strict descendant with sufficient parent Votes and an authority derived from verified evidence.

Under these assumptions, the result constructs a finite extension of the same execution that finalizes the selected valid Block and a valid strict descendant. Both finalized Blocks occur in one resulting state, advancing finalized history beyond the selected Block. The extension permits arbitrarily long finite message delays. Liveness requires continued finalization above every common reference height, as defined in Section 1.

5 Related research

5.1 Finalization

Chained HotStuff uses a three-chain commit rule under partial synchrony [2, Sections 4–5]. Its leader coordinates the Votes that establish successive quorum certificates. Tsampi applies Q1 three times, restarting from each returned Block. Each query accumulates distinct Validators' stake along that Block's ancestry under the authority recorded there.

5.2 Permitted Votes

Tendermint orders proposals through rounds with prevote and precommit steps [3, Section III, Algorithm 1]. Its locks constrain which proposal a validator may support in a later round. Tsampi checks PrevSelfVote and the ancestry of historical quorum results before permitting a later Vote. PrevSelfVote identifies the earlier signed choice against which the improvement rule is checked.

Casper FFG defines slashing conditions for Votes between source and target checkpoints [4, Section 2]. Its accountable Safety theorem connects conflicting finalized checkpoints to violations by at least one third of the total deposit. Tsampi checks conflicts between authenticated Votes and applies verified evidence to a child Block's authority. Tsampi uses the parent authority for admission, preserving historical Vote weights while verified evidence determines the child's authority.

5.3 Delivery

Narwhal separates transaction dissemination from ordering, and Tusk provides asynchronous consensus over the resulting directed acyclic graph [5, Sections 3 and 5]. Narwhal's certificates of availability establish that honest participants retain the referenced transaction data. Tsampi describes single-parent Block lineages and assumes eventual delivery of required Blocks and Votes. Historical quorum queries require the parent Votes and exact Block ancestry, so delivery includes those references.

References

[1] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. Consensus in the presence of partial synchrony. *Journal of the ACM*, 35(2):288–323, 1988. URL `https://groups.csail.mit.edu/tds/papers/Lynch/jacm88.pdf`.

[2] Maofan Yin, Dahlia Malkhi, Michael K. Reiter, Guy Golan Gueta, and Ittai Abraham. HotStuff: BFT consensus in the lens of blockchain. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*, pages 347–356, 2019. doi: 10.1145/3293611.3331591. URL `https://arxiv.org/pdf/1803.05069v6`.

[3] Ethan Buchman, Jae Kwon, and Zarko Milosevic. The latest gossip on BFT consensus. *arXiv preprint arXiv:1807.04938*, 2019. doi: 10.48550/arXiv.1807.04938.

[4] Vitalik Buterin and Virgil Griffith. Casper the friendly finality gadget. *arXiv preprint arXiv:1710.09437*, 2019. doi: 10.48550/arXiv.1710.09437.

[5] George Danezis, Lefteris Kokoris-Kogias, Alberto Sonnino, and Alexander Spiegelman. Narwhal and tusk: A DAG-based mempool and efficient BFT consensus. In *Proceedings of the Seventeenth European Conference on Computer Systems*, pages 1–17, 2022. doi: 10.1145/3492321.3519594.